
APM & Tomcat Instrumentation

Understanding Application Performance Monitoring, instrumentation,
and how the Splunk OpenTelemetry Java agent was wired into Tomcat

CONCEPT

What is APM?

Application Performance Monitoring (APM) is the practice of observing a running application to understand how well it is performing and where it is struggling.

Response time

How long each request takes, end to end

Throughput

How many requests the app handles per second

Errors

Failed requests, exceptions, and their root cause

Resource usage

CPU, memory, threads, and JVM health

Goal: catch and diagnose problems before they reach the user.

CONCEPT

What does “instrumentation” mean?

Instrumentation is the process of adding code or agents to an application so it produces telemetry — traces, metrics, and logs — describing what it is doing.

Manual instrumentation

The developer edits application source code to add tracing calls.
Precise, but requires code changes and redeploys.

Automatic instrumentation (what we used)

A java agent (`-javaagent` flag) attaches to the JVM at startup and rewrites classes in memory to capture requests, JDBC calls, and JMX/HTTP activity. No source code changes required.

This is exactly what the Splunk OpenTelemetry Java agent jar does for Tomcat here.

What the commands actually did

1

Downloaded the agent

Fetches `splunk-otel-javaagent.jar` into `/opt/splunk-otel-javaagent/` — this jar contains the OpenTelemetry auto-instrumentation logic.

2

Disabled the old config

Commented out the existing `JAVA_OPTS` line from Exercise 2 so it would not conflict with the new one.

3

Added the java agent

Wrote a new `JAVA_OPTS` that keeps the JMX options and appends `-javaagent:/opt/splunk-otel-javaagent/splunk-otel-javaagent.jar` so it loads with the JVM.

4

Set OTel environment variables

`OTEL_SERVICE_NAME`, `OTEL_EXPORTER_OTLP_ENDPOINT`, and `OTEL_RESOURCE_ATTRIBUTES` tell the agent what to call this service and where to send data.

5

Restarted Tomcat and sent a request

`systemctl restart tomcat` reloads the JVM with the agent attached; the `curl` call generates one real HTTP request, and therefore one real trace.

The OpenTelemetry environment variables

These variables are read by the java agent at JVM startup and control what it reports and where it sends it.

Variable	Value in this lab	What it means
OTEL_SERVICE_NAME	tomcat-lab	Name shown for this service in the Splunk APM UI — lets you tell it apart from other services.
OTEL_EXPORTER_OTLP_ENDPOINT	http://localhost:4318	Address the agent sends trace data to, using the OTLP protocol over HTTP.
OTEL_RESOURCE_ATTRIBUTES	deployment.environment=lab	Extra metadata tag attached to every trace, used to filter by environment (lab, dev, prod, etc.).

Why these specific port numbers?

Each port is a separate, standard communication channel — they are not interchangeable.

Port	Used for	Why this number
8080	Tomcat HTTP connector	Tomcat's long-standing default web port, set in server.xml; unrelated to monitoring.
9012	JMX remote (JVM monitoring)	An arbitrary but fixed port chosen for this lab so a JMX client can query JVM stats (memory, threads) remotely; not an OTel standard.
4318	OTLP HTTP receiver	The official default port reserved for the OpenTelemetry Protocol (OTLP) over HTTP, defined by the OpenTelemetry spec. Port 4317 is the equivalent for OTLP over gRPC.

What does Splunk support here?

Splunk Observability Cloud (Splunk APM) is built on the open-source OpenTelemetry (OTel) standard, so instrumentation is vendor-neutral.

OpenTelemetry native

Uses the same OTel APIs, SDKs, and wire protocol used across the industry — no proprietary agent format.

Auto-instrumentation agents

Ready-made java agents for Java, .NET, Node.js, Python, and Go — attach with a flag, no code changes.

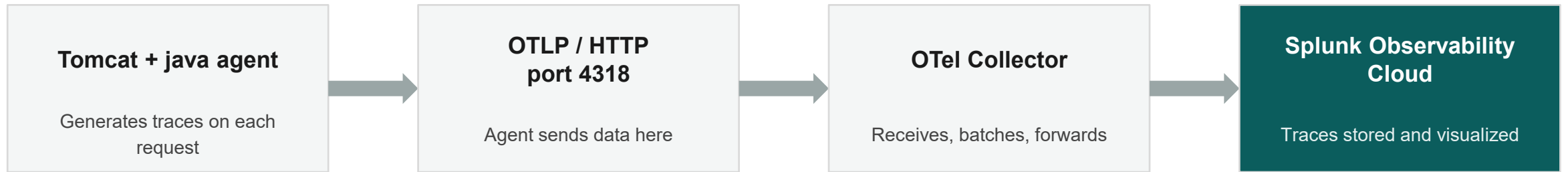
Distributed tracing

Follows one request across every service it touches, building a full timeline and service map.

Metrics and logs correlation

Ties traces to related metrics and logs so root cause analysis stays in one place.

Where the trace data goes



The curl request in the last step is what triggers this whole chain once, end to end.

RECAP

Summary

APM	Ongoing visibility into an app's speed, errors, and resource use.
Instrumentation	The mechanism — manual or automatic — that produces that visibility data.
Splunk OTel java agent	Zero-code, automatic instrumentation attached to Tomcat via <code>-javaagent</code> .
Ports (8080 / 9012 / 4318)	Three separate channels: app traffic, JVM monitoring, and telemetry export.